

Étale Cohomology of Concurrent Programs

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 22, 2026

Abstract

We develop an étale cohomology theory for concurrent programs, showing that cohomological invariants of the *semantic site* $\mathcal{S}_{\mathcal{P}}$ detect concurrency-specific obstructions invisible to the sequential (Zariski) topology. The central results are: (i) the first étale cohomology group $H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}}, \text{GL}_1)$ classifies race conditions up to synchronisation equivalence, and (ii) the Brauer group $\text{Br}(\mathcal{S}_{\mathcal{P}}) = H_{\text{ét}}^2(\mathcal{S}_{\mathcal{P}}, \mathbb{G}_m)$ classifies deadlock equivalence classes. Both groups are computable in polynomial time from the *thread interaction complex* $K_{\mathcal{P}}$, a finite simplicial complex built from the program’s thread–variable access pattern. We apply these results to a cohomological synchronisation synthesis algorithm: given a program $\mathcal{P}$ with non-trivial $H_{\text{ét}}^1$ or $\text{Br}(\mathcal{S}_{\mathcal{P}})$, the algorithm inserts synchronisation primitives that eliminate the offending cohomology classes, yielding a provably race-free and deadlock-free program. The theory embeds naturally in the JU-GEO framework as the concurrency analogue of the Grothendieck obstruction theory of Papers 3 and 6.

Contents

1	Introduction	2
2	Concurrent Programs as Semantic Sites	3
3	The Étale Topology	3
4	Sheaves and Čech Cohomology	4
5	First Étale Cohomology and Race Conditions	4
6	The Brauer Group and Deadlock Classification	5
7	The Thread Interaction Complex	6
8	Synchronisation Synthesis	6
9	Worked Examples	7
9.1	Producer–Consumer	7
9.2	Dining Philosophers	7
9.3	Three-Thread Reader–Writer	8
10	Conclusion	8

1 Introduction

Concurrent programs exhibit failure modes absent in sequential code. Race conditions arise when two threads access a shared variable without synchronisation; deadlocks arise when threads wait for each other’s resources in a cycle. Classical analyses treat these as separate phenomena with ad-hoc fixes. This paper unifies them under a single cohomological framework rooted in Grothendieck’s étale topology.

The key observation is that a concurrent program $\mathcal{P}$ carries *two natural Grothendieck topologies* on its semantic site. The **Zariski topology** J_{Zar} models sequential execution: covering families are point-like (one global state at a time), and cohomology is identically zero—no concurrency structure is visible. The **étale topology** $J_{\text{ét}}$ models concurrent execution: covering families are *unramified decompositions*—collections of thread-local views that together faithfully represent the global state without aliasing. The étale topology is strictly finer than J_{Zar} , and the extra cohomology it admits is exactly the space of concurrency obstructions.

Main results.

Theorem A (Race classification).

$H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}}, \text{GL}_1) \cong \text{Race}(\mathcal{P})/\sim_{\text{sync}}$: the first étale cohomology with GL_1 coefficients classifies race conditions up to synchronisation equivalence. $\mathcal{P}$ is race-free iff $H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}}, \text{GL}_1) = 0$.

Theorem B (Deadlock classification).

$\text{Br}(\mathcal{S}_{\mathcal{P}}) = H_{\text{ét}}^2(\mathcal{S}_{\mathcal{P}}, \mathbb{G}_m) \cong \text{Dead}(\mathcal{P})/\sim_{\text{dead}}$: the Brauer group classifies deadlock equivalence classes. $\mathcal{P}$ is deadlock-free iff $\text{Br}(\mathcal{S}_{\mathcal{P}}) = 0$.

Theorem C (Computability).

Both groups are isomorphic to simplicial cohomology of $K_{\mathcal{P}}$, computable in $O(n^3)$ time.

Algorithm D (Synthesis).

Non-trivial cohomology classes are eliminated by inserting mutexes ($H_{\text{ét}}^1$) and resource orderings (Br).

Contributions.

1. A *semantic site* $\mathcal{S}_{\mathcal{P}}$ for concurrent programs with Zariski and étale topologies.
2. *Theorems A and B*: cohomological classification of races and deadlocks via $H_{\text{ét}}^1$ and the Brauer group.
3. *Theorem C*: polynomial-time computability via $K_{\mathcal{P}}$ and the Leray spectral sequence.
4. *Algorithm D*: minimal synchronisation synthesis from cohomological obstructions.
5. *Lean 4 formalization* in `Paper26_EtaleCohomology.lean`.

Organization. Sections 2–4 build the site and cohomology. Sections 5–6 prove Theorems A and B. Section 7 proves Theorem C. Sections 8–9 present Algorithm D and examples.

2 Concurrent Programs as Semantic Sites

Definition 2.1 (Concurrent program). A *concurrent program* is a tuple $\mathcal{P} = (\mathbf{T}, \mathbf{X}, \{D_i\}, \{A_i\}, \Sigma)$ where:

- $\mathbf{T} = \{T_1, \dots, T_n\}$ is a finite set of *threads*;
- $\mathbf{X} = \{x_1, \dots, x_m\}$ is a finite set of *shared variables*;
- $D_i \subseteq \mathbf{X}$ is the *domain* of T_i ;
- $A_i : D_i \rightarrow \{\mathbf{R}, \mathbf{W}, \mathbf{RW}\}$ is the *access mode*; and
- $\Sigma = \text{Val}^{\mathbf{X}}$ is the *global state space*.

Definition 2.2 (Semantic category $\mathcal{C}_{\mathcal{P}}$). The *semantic category* of $\mathcal{P}$ has:

- **Objects.** *Execution views* $U_{i,\sigma} = (T_i, \sigma)$ for each thread T_i and local state $\sigma : D_i \rightarrow \text{Val}$, plus *global objects* $U_{*,\sigma}$ for $\sigma \in \Sigma$.
- **Morphisms.** Restriction maps $\rho_i : U_{*,\sigma} \rightarrow U_{i,\sigma|_{D_i}}$, synchronisation maps $\iota_i : U_{i,\tau} \rightarrow U_{*,\sigma}$ when $\sigma[\tau] = \sigma$, and refinement maps $U_{i,\tau'} \rightarrow U_{i,\tau}$.
- **Composition.** Standard, associative, with identities.

Definition 2.3 (Semantic site). The *semantic site* $\mathcal{S}_{\mathcal{P}} = (\mathcal{C}_{\mathcal{P}}, J)$ is $\mathcal{C}_{\mathcal{P}}$ with a Grothendieck topology J . The *Zariski topology* J_{Zar} admits only trivial covers of global objects $U_{*,\sigma}$.

Proposition 2.4 (Zariski cohomology vanishes). *For every abelian sheaf $\mathcal{F}$ on $(\mathcal{S}_{\mathcal{P}}, J_{\text{Zar}})$, $H^n(\mathcal{S}_{\mathcal{P}}, \mathcal{F}) = 0$ for all $n \geq 1$.*

Proof. J_{Zar} is indiscrete on global objects; every cover is a singleton. The Čech complex has $\check{C}^n = 0$ for $n \geq 1$, so all higher Čech groups vanish. $\square$

3 The Étale Topology

In algebraic geometry, étale morphisms generalise local isomorphisms (formally étale + locally of finite presentation). For concurrent programs, the analogue is a thread-local view that faithfully represents the global state without aliasing.

Definition 3.1 (Étale morphism). A morphism $f : U' \rightarrow U$ in $\mathcal{C}_{\mathcal{P}}$ is *étale* if:

1. (*Same-thread or sync*) $U' = U_{i,\tau'}$, $U = U_{i,\tau}$ for the same T_i , or $U' = U_{i,\tau}$, $U = U_{*,\sigma}$ with $\tau = \sigma|_{D_i}$;
2. (*Unramified*) No two threads in any étale cover hold conflicting values of any shared variable; and
3. (*Locally of finite presentation*) The transition is described by finitely many variable updates.

Definition 3.2 (Étale topology $J_{\text{ét}}$). A family $\{f_{\alpha} : U_{\alpha} \rightarrow U\}$ is an *étale cover* if each f_{α} is étale, the family is jointly surjective, and it is non-aliasing (at most one thread writes any variable x). The *étale site* $(\mathcal{C}_{\mathcal{P}}, J_{\text{ét}})$ uses the Grothendieck topology generated by étale covers.

Proposition 3.3 (Étale strictly refines Zariski). $J_{\text{Zar}} \subseteq J_{\text{ét}}$, strictly when $n \geq 2$ threads share a variable.

Proof. Every trivial cover is étale. Strictness: if T_i and T_j both read x ($i \neq j$), the family $\{U_{i,\sigma|D_i} \rightarrow U_{*,\sigma}, U_{j,\sigma|D_j} \rightarrow U_{*,\sigma}\}$ is étale (reads, non-aliasing) but not a Zariski cover. $\square$

Remark 3.4 (Geometric analogy). Execution points of $\mathcal{S}_{\mathcal{P}}$ are pairs $(T_i, v : D_i \rightarrow \text{Val})$. The étale fundamental group $\pi_1^{\text{ét}}(\mathcal{S}_{\mathcal{P}}, p)$ classifies étale covering spaces, corresponding to globally consistent locking disciplines. A race is an execution point where the étale covering fails: two thread-views diverge over a shared variable.

4 Sheaves and Čech Cohomology

Definition 4.1 (Coefficient sheaves). The *structure sheaf* assigns $\mathcal{O}_{\mathcal{S}}(U_{i,\tau}) = \mathbb{Z}[\tau(D_i)]$. The *multiplicative group sheaf* $\mathbb{G}_m(U) = \mathcal{O}_{\mathcal{S}}(U)^\times$. The GL_1 -sheaf assigns to $U_{i,\tau}$ the group of invertible synchronisation tokens.

Definition 4.2 (Čech cohomology). For an étale cover $\mathfrak{U} = \{U_\alpha \rightarrow U\}$ and sheaf $\mathcal{F}$:

$$\check{C}^n(\mathfrak{U}, \mathcal{F}) = \prod_{\alpha_0 < \dots < \alpha_n} \mathcal{F}(U_{\alpha_0} \times_U \dots \times_U U_{\alpha_n}),$$

with the alternating coboundary δ^n . The *étale cohomology* $H_{\text{ét}}^n(\mathcal{S}_{\mathcal{P}}, \mathcal{F}) = \varinjlim_{\mathfrak{U}} \check{H}^n(\mathfrak{U}, \mathcal{F})$.

Proposition 4.3 (Čech-to-derived). *There is a convergent spectral sequence $\check{H}^p(\mathcal{S}_{\mathcal{P}}, \mathcal{H}^q(\mathcal{F})) \Rightarrow H_{\text{ét}}^{p+q}(\mathcal{S}_{\mathcal{P}}, \mathcal{F})$. For sheaves acyclic on affines, $\check{H}^n \cong H_{\text{ét}}^n$.*

GL_1 and $\mathbb{G}_m$ are acyclic on $U_{i,\tau}$ (local rings; analogous to Hilbert 90); we compute étale cohomology via Čech complexes.

5 First Étale Cohomology and Race Conditions

Definition 5.1 (Race condition). A *race condition* is a triple $r = (T_i, T_j, x)$ with $i \neq j$, $x \in D_i \cap D_j$, and at least one access mode is a write. Let $\text{Race}(\mathcal{P})$ be the set of all race conditions.

Definition 5.2 (Synchronisation equivalence). Races $r = (T_i, T_j, x)$ and $r' = (T_{i'}, T_{j'}, x')$ are *synchronisation-equivalent* ($r \sim_{\text{Sync}} r'$) if $i = i'$, $j = j'$, $x = x'$. The quotient $\text{Race}(\mathcal{P})/\sim_{\text{Sync}}$ is a free abelian group.

Theorem 5.3 (Race classification). *There is a natural isomorphism of abelian groups*

$$H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}}, \text{GL}_1) \cong \text{Race}(\mathcal{P})/\sim_{\text{Sync}}.$$

In particular, $\mathcal{P}$ is race-free iff $H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}}, \text{GL}_1) = 0$.

Proof sketch. Fix $\mathfrak{U}_\sigma = \{U_{i,\sigma|D_i} \rightarrow U_{*,\sigma}\}_{i=1}^n$.

(Cocycles are synchronisation schemes.) A GL_1 -1-cochain assigns to each overlap $U_{ij} = U_{i,\sigma|D_i} \times_{U_{*,\sigma}} U_{j,\sigma|D_j}$ a token $c_{ij} \in \text{GL}_1(U_{ij})$. The cocycle condition $c_{ij} \cdot c_{jk} = c_{ik}$ requires consistent pairwise synchronisation.

(Coboundaries are global schemes.) A coboundary $c_{ij} = b_i \cdot b_j^{-1}$ comes from a global assignment $\{b_i\}$: a consistent global locking scheme.

(Non-trivial classes are races.) When T_i and T_j both write to x , the overlap U_{ij} witnesses aliasing. The token c_{ij} cannot be split as b_i/b_j (no consistent global state for x), so $[c_{ij}] \neq 0$. Each race (T_i, T_j, x) gives a distinct class in $H_{\text{ét}}^1$; $\sim_{\text{Sync}}$ -equivalent races give the same class. $\square$

Corollary 5.4 (Hilbert 90 for read-only threads). *If all threads are read-only, $H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}}, \text{GL}_1) = 0$: the concurrent analogue of Hilbert’s Theorem 90.*

6 The Brauer Group and Deadlock Classification

Definition 6.1 (Deadlock). A *deadlock of length $k \geq 2$* is a sequence $(T_{i_1}, \dots, T_{i_k}, R_1, \dots, R_k)$ such that T_{i_ℓ} holds R_ℓ and waits for $R_{\ell+1} \pmod k$, each R_ℓ held exclusively. Let $\text{Dead}(\mathcal{P})$ denote all deadlocks. Deadlocks are *deadlock-equivalent* if resolved by the same resource ordering.

Definition 6.2 (Azumaya algebra). An *Azumaya algebra* on $\mathcal{S}_{\mathcal{P}}$ is a sheaf of $\mathcal{O}_{\mathcal{S}}$ -algebras $\mathcal{A}$ étale-locally isomorphic to $M_n(\mathcal{O}_{\mathcal{S}})$. The *Brauer group* $\text{Br}(\mathcal{S}_{\mathcal{P}}) = H_{\text{ét}}^2(\mathcal{S}_{\mathcal{P}}, \mathbb{G}_m)$ is the group of Morita equivalence classes of Azumaya algebras.

A twisted Azumaya algebra (locally $M_n(\mathcal{O}_{\mathcal{S}})$, globally non-trivial) represents a resource pool with a cyclic allocation dependency: a deadlock.

Theorem 6.3 (Brauer–deadlock correspondence). *There is a natural isomorphism*

$$\text{Br}(\mathcal{S}_{\mathcal{P}}) = H_{\text{ét}}^2(\mathcal{S}_{\mathcal{P}}, \mathbb{G}_m) \cong \text{Dead}(\mathcal{P}) / \sim_{\text{dead}}.$$

In particular, $\mathcal{P}$ is deadlock-free iff $\text{Br}(\mathcal{S}_{\mathcal{P}}) = 0$.

Proof sketch. Consider the exact sequence of sheaves

$$1 \rightarrow \mathbb{G}_m \xrightarrow{n} \text{GL}_n \xrightarrow{\pi} \text{PGL}_n \rightarrow 1. \quad (1)$$

The connecting map $\delta : H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}}, \text{PGL}_n) \rightarrow H_{\text{ét}}^2(\mathcal{S}_{\mathcal{P}}, \mathbb{G}_m)$ sends an Azumaya algebra $\mathcal{A}$ to its Brauer class.

(Deadlocks give non-trivial Brauer classes.) A deadlock cycle $(T_{i_1}, \dots, T_{i_k})$ defines a cyclic ordering of $\{R_1, \dots, R_k\}$. The associated 2-cocycle $c_{\ell, \ell+1, \ell+2} \in \mathbb{G}_m(U_{i_\ell, i_{\ell+1}, i_{\ell+2}})$ is not a coboundary: $\prod_{\ell} c_{\ell, \ell+1, \ell+2} \neq 1$.

(Non-trivial Brauer classes are deadlocks.) A non-trivial $[\mathcal{A}]$ means resources are locally but not globally orderable. By Dilworth’s theorem, failure to extend local total orders implies a circular dependency: a deadlock cycle. $\square$

Corollary 6.4 (Torsion order equals cycle length). *$\text{Br}(\mathcal{S}_{\mathcal{P}})$ is torsion; the order of $[\mathcal{A}]$ equals the length of the shortest deadlock in that class.*

The long exact sequence from (1) gives an *escalation* map $\delta : \text{Race}(\mathcal{P}) / \sim_{\text{Sync}} \rightarrow \text{Dead}(\mathcal{P}) / \sim_{\text{dead}}$: a race naively fixed by an unordered mutex can escalate to a deadlock.

7 The Thread Interaction Complex

Definition 7.1 (Thread interaction complex). $K_{\mathcal{P}}$ is the simplicial complex with:

- **0-simplices:** $\{T_1, \dots, T_n\}$;
- **1-simplices:** $\{T_i, T_j\}$ when $(T_i, T_j, x) \in \text{Race}(\mathcal{P})$; and
- **k -simplices** ($k \geq 2$): $\{T_{i_0}, \dots, T_{i_k}\}$ for $(k+1)$ -way deadlock cycles.

The 1-skeleton $G_{\mathcal{P}}$ is the *thread interaction graph*.

Theorem 7.2 (Comparison theorem). *For all $n \geq 1$:*

$$H_{\text{ét}}^n(\mathcal{S}_{\mathcal{P}}, \mathbb{G}_m) \cong H^n(K_{\mathcal{P}}, \mathbb{Z}).$$

In particular,

$$H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}}, \text{GL}_1) \cong \mathbb{Z}^{\beta_1(G_{\mathcal{P}})}, \quad (2)$$

$$\text{Br}(\mathcal{S}_{\mathcal{P}}) \cong H^2(K_{\mathcal{P}}, \mathbb{Z}), \quad (3)$$

where $\beta_1(G_{\mathcal{P}}) = |E(G_{\mathcal{P}})| - |V(G_{\mathcal{P}})| + c(G_{\mathcal{P}})$.

Proof sketch. Apply the Leray spectral sequence for $\pi : \mathcal{S}_{\mathcal{P}} \rightarrow |K_{\mathcal{P}}|$:

$$E_2^{p,q} = H^p(|K_{\mathcal{P}}|, R^q \pi_* \mathbb{G}_m) \implies H_{\text{ét}}^{p+q}(\mathcal{S}_{\mathcal{P}}, \mathbb{G}_m).$$

Each fiber of π is contractible and affine, so $R^q \pi_* \mathbb{G}_m = 0$ for $q \geq 1$. The sequence degenerates: $H_{\text{ét}}^n(\mathcal{S}_{\mathcal{P}}, \mathbb{G}_m) \cong H^n(|K_{\mathcal{P}}|, \mathbb{Z})$.

Equation (2): H^1 of a graph is $\mathbb{Z}^{\beta_1}$. Equation (3): $H^2(K_{\mathcal{P}}, \mathbb{Z})$ is generated by 2-simplices (3-way deadlock triples). $\square$

Corollary 7.3 (Polynomial computability). *Both $H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}}, \text{GL}_1)$ and $\text{Br}(\mathcal{S}_{\mathcal{P}})$ are computable in $O(n^3)$ time from $\mathcal{P}$.*

Proof. Build $K_{\mathcal{P}}$ ($O(n^2 m)$); compute β_1 via BFS ($O(n^2)$); compute H^2 via Smith normal form ($O(n^3)$). $\square$

8 Synchronisation Synthesis

Given non-trivial cohomology, we synthesise synchronisation primitives to reduce it to zero—the JUGEOREPAIR semantic move for concurrency.

Definition 8.1 (Cohomology elimination). A *mutex insertion* for a race class α corresponding to (T_i, T_j, x) adds mutex m_x on x . After insertion, the cocycle c_{ij} becomes b_i/b_j (global token m_x for all threads accessing x); hence $[\alpha] = 0$.

A *resource ordering* for a Brauer class β imposes a total order on $\mathbf{X}$. All threads acquire resources in increasing index order; the wait-for graph becomes acyclic, removing all 2-simplices: $\text{Br}(\mathcal{S}_{\mathcal{P}}) = 0$.

Theorem 8.2 (Synthesis correctness). *Algorithm D terminates and produces $\mathcal{P}'$ with $H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}'}, \text{GL}_1) = 0$ and $\text{Br}(\mathcal{S}_{\mathcal{P}'}) = 0$.*

Proof. Each mutex insertion removes one cycle edge from $G_{\mathcal{P}}$, reducing β_1 by at most 1. After eliminating all β_1 generators, the graph is a forest. Each resource ordering removes 2-simplices from $K_{\mathcal{P}}$, eliminating H^2 generators. Both processes terminate (finite generators). $\square$

Algorithm D: Cohomological Synchronisation Synthesis

1. **Input:** concurrent program $\mathcal{P}$.
2. Build $K_{\mathcal{P}}$ from $\{A_i\}$.
3. Compute $H^1(K_{\mathcal{P}}, \mathbb{Z})$ and $H^2(K_{\mathcal{P}}, \mathbb{Z})$.
4. **For each** generator α_k of H^1 :
 - (a) Locate cycle edge $\{T_i, T_j\}$ in $G_{\mathcal{P}}$ for α_k .
 - (b) Insert mutex m_{x_k} on each variable causing the race.
 - (c) Update $K_{\mathcal{P}}$; recompute H^1 .
5. **For each** generator β_ℓ of H^2 :
 - (a) Locate 2-simplex $\{T_i, T_j, T_k\}$ for β_ℓ .
 - (b) Impose resource ordering (increasing index).
 - (c) Update $K_{\mathcal{P}}$; recompute H^2 .
6. **Output:** $\mathcal{P}'$ with synchronisation primitives.

Remark 8.3 (Minimality). One primitive per generator: distinct cohomology classes are linearly independent, so no single primitive eliminates two independent classes.

9 Worked Examples

9.1 Producer–Consumer

Thread T_1 writes buffer x ; T_2 reads x .

$G_{\mathcal{P}}$: two vertices, one edge. $\beta_1 = 1 - 2 + 1 = 0$; $H_{\text{ét}}^1 \cong \mathbb{Z}^0 = 0$. No 2-simplices: $\text{Br} = 0$.

The single race (T_1, T_2, x) is cohomologically simple (tree graph) and handled by one mutex m_x . After insertion: $H_{\text{ét}}^1 = 0$, $\text{Br} = 0$.

$\beta_1 = 0$ despite a race because the graph is a tree: the race is independently resolvable with one mutex and creates no cyclic dependency.

9.2 Dining Philosophers

Five philosophers $T_1, \dots, T_5$; five forks $R_0, \dots, R_4$. T_ℓ needs $R_{\ell-1}$ and $R_\ell \pmod{5}$; all modes RW.

$G_{\mathcal{P}} = C_5$: five vertices, five edges, one component. $\beta_1(C_5) = 5 - 5 + 1 = 1$; $H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}}, \text{GL}_1) \cong \mathbb{Z}$. One independent race class: circular lock contention.

Deadlock under naive fix. If each philosopher grabs the left fork first, the wait-for graph is C_5 . Philosopher-triples give 2-simplices: $H^2(K_{\mathcal{P}}, \mathbb{Z}) \cong \mathbb{Z}/5\mathbb{Z}$, $\text{Br}(\mathcal{S}_{\mathcal{P}}) \cong \mathbb{Z}/5\mathbb{Z}$ (5-torsion).

Synthesis. Algorithm D imposes ordering $R_0 < R_1 < \dots < R_4$. Philosopher T_5 grabs R_0 before R_4 , breaking the cycle. Result: $\beta_1 = 0$, $H^2 = 0$, $H_{\text{ét}}^1 = 0$, $\text{Br} = 0$.

9.3 Three-Thread Reader–Writer

T_1 reads x ; T_2 writes x , reads y ; T_3 reads x , writes y .

Races: (T_1, T_2, x) , (T_1, T_3, x) , (T_2, T_3, y) . $G_{\mathcal{P}} = K_3$. The 2-simplex $\{T_1, T_2, T_3\}$ is present (all three mutually interact). $K_{\mathcal{P}} = \Delta^2$ (filled triangle), contractible: $H^1(\Delta^2, \mathbb{Z}) = 0$, $H^2(\Delta^2, \mathbb{Z}) = 0$.

$H_{\text{ét}}^1 = 0$ and $\text{Br} = 0$: despite a 1-cycle in K_3 , the filled 2-simplex makes $K_{\mathcal{P}}$ contractible. A single read–write lock on $\{x, y\}$ suffices with no deadlock risk.

10 Conclusion

We have unified race conditions and deadlocks as cohomological obstructions in a two-dimensional theory: $H_{\text{ét}}^1(\mathcal{S}_{\mathcal{P}}, \text{GL}_1)$ classifies races and $\text{Br}(\mathcal{S}_{\mathcal{P}}) = H_{\text{ét}}^2(\mathcal{S}_{\mathcal{P}}, \mathbb{G}_m)$ classifies deadlocks. Both reduce to simplicial cohomology of $K_{\mathcal{P}}$ and are computable in polynomial time. The cohomological structure drives Algorithm D, a minimal synchronisation synthesis procedure.

The framework is an instantiation of the JUGEO obstruction theory: étale cohomology groups are the obstruction bundle $\mathcal{B}$ of Paper 3 specialised to the concurrent site; Algorithm D is the REPAIR move of Paper 6 specialised to locking primitives.

Future directions. (i) $H_{\text{ét}}^n$ for $n \geq 3$ may classify livelock and priority inversion. (ii) Motivic refinements of $\mathbb{G}_m$ may give finer race invariants. (iii) Equivariant Brauer groups $\text{Br}^G(\mathcal{S}_{\mathcal{P}})$ for symmetric thread pools. (iv) Full integration of Algorithm D into the JUGEO pipeline.

References

- [1] M. Artin, A. Grothendieck, J.-L. Verdier. *Théorie des topos et cohomologie étale des schémas (SGA 4)*. Lecture Notes in Math. 269, 270, 305. Springer, 1972–1973.
- [2] J. S. Milne. *Étale Cohomology*. Princeton Mathematical Series 33. Princeton University Press, 1980.
- [3] A. Grothendieck. Le groupe de Brauer I–III. In *Dix Exposés sur la Cohomologie des Schémas*, pp. 46–188. North-Holland, 1968.
- [4] E. W. Dijkstra. Hierarchical ordering of sequential processes. *Acta Informatica*, 1(2):115–138, 1971.
- [5] C. Flanagan, S. N. Freund. FastTrack: Efficient and precise dynamic race detection. In *PLDI*, 2009.
- [6] M. Naik, A. Aiken, J. Whaley. Effective static deadlock detection. In *ICSE*, 2006.
- [7] S. Kahrs. Sheaf models for concurrent systems. *Theoretical Computer Science*, 124(1):3–29, 1994.
- [8] É. Goubault, T. P. Jensen. Homology of higher-dimensional automata. In *CONCUR*, 1992.
- [9] L. Fajstrup, É. Goubault, E. Haucourt, S. Mimram, M. Raussen. *Directed Algebraic Topology and Concurrency*. Springer, 2016.
- [10] H. Young. Sheaf descent and proof obstructions. *Judgment Geometry*, Paper 3, 2024.
- [11] H. Young. Semantic moves and proof repair. *Judgment Geometry*, Paper 6, 2024.